

Ejercicio 7: Apadrinamiento de Animales (Ramificación y Poda)

Enunciado

Para sensibilizar a los niños sobre la protección de los animales, en un colegio se ha decidido que cada niño apadrine un animal mediante una adopción simbólica de una especie diferente. Hay n niños en el colegio y cada niño ha dado su preferencia para cada una de las n especies diferentes que se pretenden apadrinar (0 si no le gusta, 1 si le gusta).

Diseña un algoritmo para encontrar una asignación niño-animal que maximice la satisfacción global (suma de las preferencias obtenidas). Plantea el problema como un problema de mínimo y aplica el esquema de **Ramificación y Poda** de mínimo coste.

Transformación a un Problema de Minimización

Sea P una matriz de preferencias de tamaño $n \times n$ donde $P[i][j] \in \{0, 1\}$ representa la preferencia del niño i por el animal j . Queremos maximizar la satisfacción global:

$$\text{Maximizar } \sum_{i=1}^n P[i][\text{asignado}(i)]$$

Para transformarlo en un problema de **mínimo coste**, definimos una matriz de costes C de tamaño $n \times n$ donde el coste de asignar el animal j al niño i es inverso a su preferencia:

$$C[i][j] = 1 - P[i][j]$$

De este modo:

- Si al niño i le gusta el animal j ($P[i][j] = 1$), el coste es $C[i][j] = 0$.
- Si al niño i no le gusta el animal j ($P[i][j] = 0$), el coste es $C[i][j] = 1$.

Maximizar la satisfacción es matemáticamente equivalente a **minimizar el coste total**:

$$\text{Minimizar } \sum_{i=1}^n C[i][\text{asignado}(i)]$$

a) Representación de las Soluciones y Árbol del Espacio de Soluciones

Representación de una Solución

Una solución completa se representa mediante una tupla o vector $X = (x_1, x_2, \dots, x_n)$ de tamaño n , donde:

- $x_i \in \{1, 2, \dots, n\}$ representa el identificador del animal asignado al niño i .
- **Restricción de asignación única (biyección):** $x_i \neq x_j$ para todo $i \neq j$ (cada animal es apadrinado por un único niño).

Árbol del Espacio de Estados (Soluciones)

El espacio de soluciones se organiza en un árbol de búsqueda:

- **Nivel del árbol:** Cada nivel i (desde 1 hasta n) corresponde a la decisión de asignar un animal al niño i .
- **Nodo Raíz (Nivel 0):** Representa el estado inicial en el que ningún niño tiene asignado ningún animal: $X = (\emptyset, \dots, \emptyset)$.
- **Nodo Intermedio en el Nivel k :** Representa una solución parcial donde se han asignado animales a los primeros k niños:

$$X_k = (x_1, x_2, \dots, x_k) \quad \text{con } x_j \text{ distintos entre sí}$$

- **Ramificación:** Un nodo en el nivel k genera sus hijos en el nivel $k + 1$ asignando al niño $k + 1$ cualquiera de los animales que **aún no hayan sido asignados** en los niveles anteriores. El factor de ramificación de un nodo en el nivel k es exactamente de $n - k$ hijos.
- **Nodos Hojas (Nivel n):** Representan asignaciones completas y válidas de los n animales a los n niños (permutaciones del conjunto $\{1, \dots, n\}$).

b) Función de Coste Real $c(x)$

Para cualquier nodo x del árbol de búsqueda (sea intermedio o una hoja), la función de coste real $c(x)$ representa el coste de la **mejor solución completa** (u óptima) que se puede alcanzar descendiendo por la rama de ese nodo.

Si el nodo x está en el nivel k con una asignación parcial $(x_1, \dots, x_k)$, el coste de cualquier hoja descendiente completada $X = (x_1, \dots, x_k, x_{k+1}, \dots, x_n)$ se define como:

$$c(X) = \sum_{i=1}^n C[i][x_i]$$

c) Función de Coste Estimado $\hat{c}(x)$ (Cota Inferior / Lower Bound)

La función de coste estimado $\hat{c}(x)$ calcula una **cota inferior (optimista)** del coste mínimo que se puede obtener a partir del nodo x . Para que el algoritmo sea correcto, debe cumplirse la propiedad de admisibilidad: $\hat{c}(x) \leq c(x)$.

Para un nodo x en el nivel k con asignación parcial $(x_1, \dots, x_k)$, definimos el conjunto de animales ya asignados como $A_k = \{x_1, \dots, x_k\}$ y los animales libres como $L_k = \{1, \dots, n\} \setminus A_k$.

La cota inferior se calcula como la suma del coste ya acumulado más una estimación optimista para los niños restantes:

$$\hat{c}(x) = \sum_{i=1}^k C[i][x_i] + \sum_{i=k+1}^n \min_{j \in L_k} C[i][j]$$

Justificación de la Cota Inferior

- El primer sumando es el coste exacto e inevitable de las decisiones ya tomadas para los primeros k niños.
- El segundo sumando asume el escenario ideal (optimista) en el que cada niño restante i (desde $k + 1$ hasta n) puede quedarse con su animal preferido de entre los que quedan libres, **ignorando la restricción de que dos niños restantes no puedan elegir el mismo animal**.
- Como esta estimación asume un problema relajado (permite colisiones de asignación en los pasos futuros), el valor obtenido siempre será menor o igual que el coste de cualquier asignación real completa y válida. Por tanto, es una cota inferior válida ($\hat{c}(x) \leq c(x)$).

d) Función de Poda $U(x)$ (Cota Superior / Upper Bound)

La función de poda se apoya en una **cota superior global** U que almacena el coste de la mejor solución completa encontrada hasta el momento durante la exploración.

Estimación de Cota Superior local $U(x)$

Para un nodo x en el nivel k , podemos calcular una cota superior viable construyendo rápidamente una solución completa (factible pero no necesariamente óptima) mediante una **estrategia voraz** sobre los elementos restantes:

1. Partimos de la asignación parcial del nodo x .
2. Para cada niño restante $i = k + 1 \dots n$, le asignamos de forma voraz el animal disponible $j \in L_k$ que minimice el coste inmediato $C[i][j]$, marcando dicho animal como ocupado antes de pasar al siguiente niño.
3. El coste de la solución completa resultante de esta asignación rápida es nuestra cota superior local $U(x)$.

Criterio de Poda

1. **Poda:** Un nodo x es **podado** (se descarta y no se ramifica) si su coste estimado optimista supera o iguala la mejor solución ya descubierta:

$$\text{Si } \hat{c}(x) \geq U \implies \text{Podar nodo } x$$